

Теория параллелизма

Отчет

Задание №6

Выполнил, Пономарев Андрей Александрович 23934
25.05.2025

Цель работы: реализовать программу для решения уравнения теплопроводности, изучить время выполнения программы на CPU на одном ядре и на нескольких, изучить время выполнения на GPU, выполнить профилирование программы с использованием “Nsight Systems”.

Используемый компилятор: gpc++

Используемый профилировщик: Nsight Systems

Замер времени: библиотека chrono

Выполнение на CPU

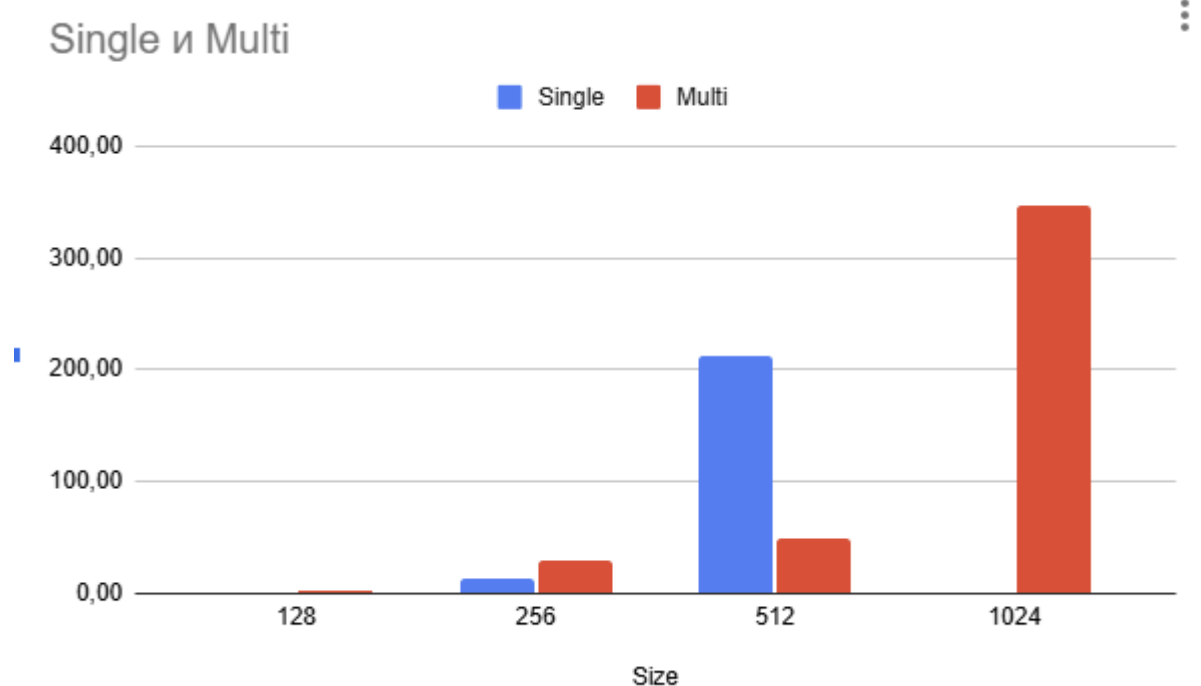
CPU-onecore

Размер сетки	Время выполнения	Точность (final error)	Количество итераций
128*128	0.920166	0.000001	30075
256*256	13.858765	0.000001	102885
512*512	211.502202	0.000001	339599

CPU-multicore

Размер сетки	Время выполнения	Точность	Количество итераций
128*128	2.446336	0.000001	30075
256*256	29.769453	0.000001	102885
512*512	49.666926	0.000001	339599
1024*1024	346.737076	0.000001	1000000

Диаграмма сравнения время работы CPU-one и CPU-multi



Выполнение на GPU

Этапы оптимизации на сетке 512*512

Этап №	Время выполнения	Точность	Максимальное количество итераций	Комментарии
1	15.71309	0.000001	1000000	первая версия
2	15.631527	0.000001	1000000	попытка внедрить collapse()
3	12.019444	0.000001	1000000	error считается не на каждом шаге

Начальный вариант:

Time (%)	Total Time (ns)	Instances	Avg (ns)	Med (ns)	Min (ns)	Max (ns)	StdDev (ns)	Style	Range
98.1	503252831	1	503252831.0	503252831.0	503252831	503252831	0.0	PushPop	init
1.0	4876794	1	4876794.0	4876794.0	4876794	4876794	0.0	PushPop	while
0.5	2546345	100	25463.5	22427.5	21609	258820	23632.4	PushPop	calc
0.4	2276244	100	22762.4	22147.0	21353	30312	1449.1	PushPop	copy

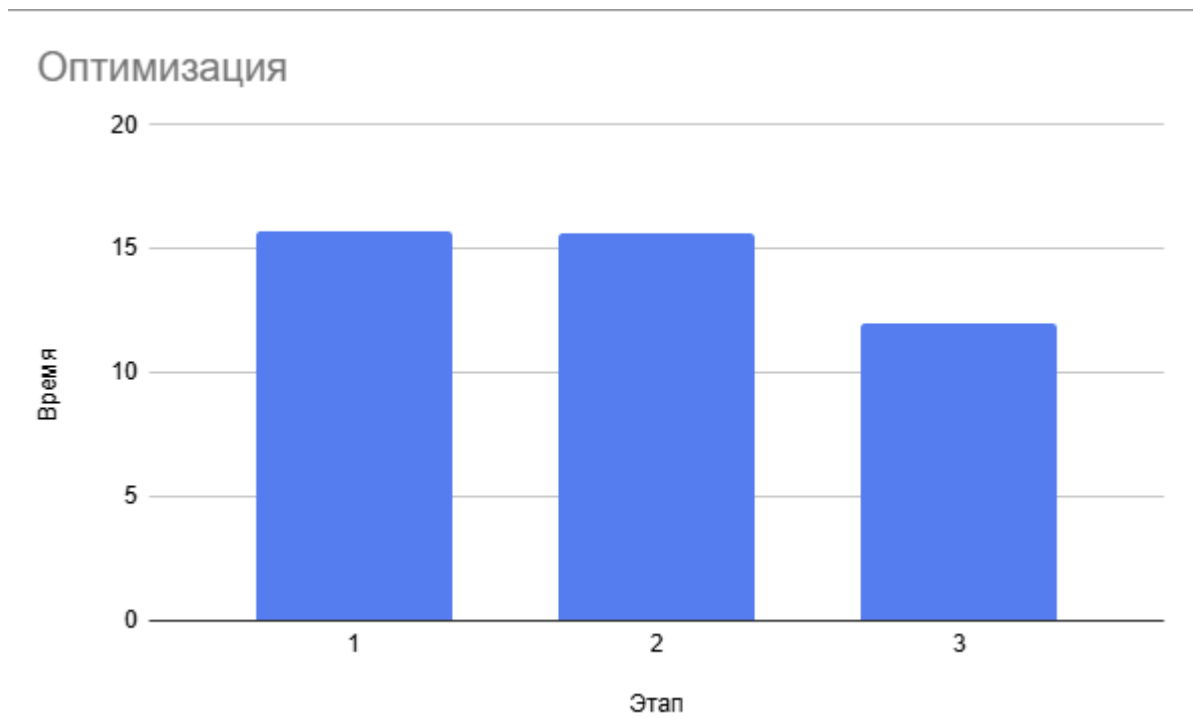
Второй этап: внедрение collapse():

Time (%)	Total Time (ns)	Instances	Avg (ns)	Med (ns)	Min (ns)	Max (ns)	StdDev (ns)	Style	Range
98.3	500165503	1	500165503.0	500165503.0	500165503	500165503	0.0	PushPop	init
0.8	4267137	1	4267137.0	4267137.0	4267137	4267137	0.0	PushPop	while
0.4	2219715	100	22197.2	20084.5	19484	215165	19509.9	PushPop	calc
0.4	2001112	100	20011.1	19841.5	19366	30053	1104.4	PushPop	copy

Третий этап: Проверка ошибки каждую 1000 итерацию

Time (%)	Total Time (ns)	Instances	Avg (ns)	Med (ns)	Min (ns)	Max (ns)	StdDev (ns)	Style	Range
98.5	565215828	1	565215828.0	565215828.0	565215828	565215828	0.0	PushPop	init
0.8	4304648	1	4304648.0	4304648.0	4304648	4304648	0.0	PushPop	while
0.4	2249081	100	22490.8	19989.5	19233	251441	23162.8	PushPop	calc
0.3	2004417	100	20044.2	19669.0	18857	30433	1707.1	PushPop	copy

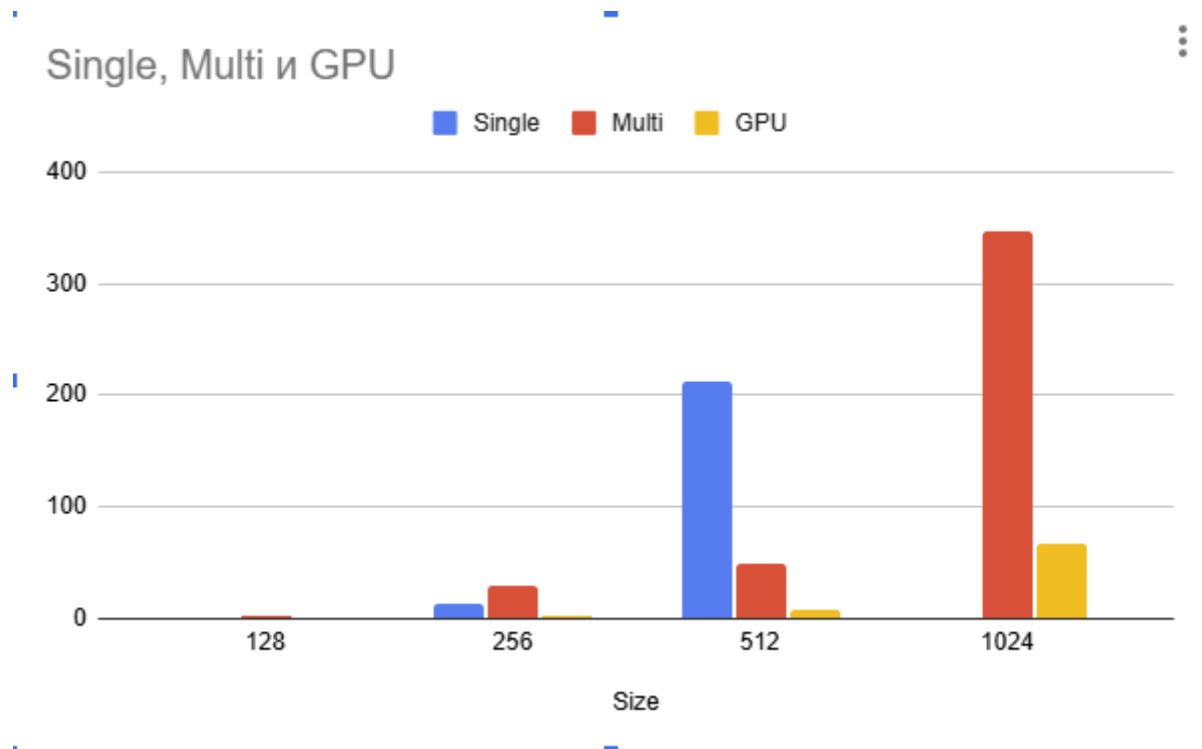
Диаграмма оптимизации



GPU – оптимизированный вариант

Размер сетки	Время выполнения	Точность	Количество итераций
128*128	0.647406	0.000001	31000
256*256	2.290811	0.000001	103000
512*512	8.616552	0.000001	340000
1024*1024	66.571557	0.000001	999999

Диаграмма сравнения времени работы CPU-one, CPU-multi, GPU(оптимизированный вариант) для разных размеров сеток



Вывод

На итоговой диаграмме видно, что распараллеливание на GPU работает быстрее всего. Большое количество физических ядер в совокупности большим объёмом видеопамати позволяют добиться реального распределения работы в этой задаче. Также хорошо видно, что чем больше сетка, тем быстрее работает GPU по сравнению с остальными. Это объясняется тем, что данные на GPU сразу уходят на обработку, а на CPU необходимо ожидать “своей очереди”. На небольших же значениях разрыв меньше. Это объясняется тем, что хотя процессор имеет небольшое количество ядер, его хватает для распараллеливания, соответственно время “ожидания в очереди” сокращается. К тому же процессорные ядра быстрее, соответственно если данных на обработку не так много, процессор в некоторых задачах может даже обгонять видеокарту.